

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамента цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №4
дисциплины «Искусственный интеллект и машинное обучение»
Вариант 12

Выполнила:
Коробка В.А.,
2 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Воронкин Роман Александрович,
доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____
Ставрополь, 2026 г.

Тема: Введение в pandas: изучение структуры Series и базовых операций

Цель: познакомить с основами работы с библиотекой pandas, в частности, со структурой данных Series

Ссылка на github: <https://github.com/13r4lera/ML4.git>

Порядок выполнения работы

Были проработаны все примеры лабораторной работы.

Было выполнено задание 1. В программе была использована библиотека pandas. Создан объект типа Series, содержащий пять целочисленных значений (5, 15, 25, 35, 45) с пользовательскими индексами a, b, c, d, e. Затем на экран были выведены содержимое серии и тип данных её элементов с помощью свойства dtype. Результат показал, что все элементы имеют целочисленный тип данных (int64).

1. Создание Series из списка

Создайте Series из списка чисел [5, 15, 25, 35, 45] с индексами ['a', 'b', 'c', 'd', 'e']. Выведите его на экран и определите его тип данных.

[2]:

```
import pandas as pd

s = pd.Series([5, 15, 25, 35, 45], index=['a', 'b', 'c', 'd', 'e'])
print(s)
print(s.dtype)
```

```
a    5
b   15
c   25
d   35
e   45
dtype: int64
int64
```

Рисунок 1. Выполнение задания 1

Было выполнено задание 2. В программе создан объект Series, содержащий числовые значения с индексами A, B, C, D, E. Для получения элемента со значением индекса C использован метод loc, а для получения элемента по его порядковому номеру использован метод iloc. В обоих случаях был получен один

и тот же элемент со значением 36, так как индекс C соответствует третьему элементу серии.

2. Получение элемента Series

Дан Series с индексами ['A', 'B', 'C', 'D', 'E'] и значениями [12, 24, 36, 48, 60].

Используйте `.loc[]` для получения элемента с индексом 'C' и `.iloc[]` для получения третьего элемента.

[6]:

```
s = pd.Series([12, 24, 36, 48, 60], index=['A', 'B', 'C', 'D', 'E'])
print(s.loc['C'])
print(s.iloc[2])
```

36

36

Рисунок 2. Выполнение задания 2

Было выполнено задание 3. В программе создан объект Series на основе массива NumPy, содержащего значения 4, 9, 16, 25, 36, 49, 64. Затем выполнена фильтрация данных с помощью логического условия $s > 20$. В результате на экран были выведены только элементы серии, значения которых превышают 20: 25, 36, 49 и 64.

3. Фильтрация данных с помощью логической индексации

Создайте Series из массива NumPy `np.array([4, 9, 16, 25, 36, 49, 64])`. Выберите только те элементы, которые больше 20, и выведите результат.

[8]:

```
import numpy as np

s = pd.Series(np.array([4, 9, 16, 25, 36, 49, 64]))
print(s[s > 20])
```

3 25

4 36

5 49

6 64

dtype: int64

Рисунок 3. Выполнение задания 3

Было выполнено задание 4. В программе создан объект Series, содержащий 50 случайных целых чисел в диапазоне от 1 до 100. Для просмотра части данных использованы методы head() и tail(). Метод head(7) выводит первые 7 элементов серии, а метод tail(5) последние 5 элементов.

4. Просмотр первых и последних элементов

Создайте Series, содержащий 50 случайных целых чисел от 1 до 100 (используйте

np.random.randint). Выведите первые 7 и последние 5 элементов с помощью .head() и .tail() .

[9]:

```
s = pd.Series(np.random.randint(1, 101, size=50))
print(s.head(7))
print(s.tail(5))
```

```
0    46
1    14
2    50
3    73
4    91
5    86
6    52
dtype: int32
45    17
46    42
47     9
48    72
49    58
dtype: int32
```

Рисунок 4. Выполнение задания 4

Было выполнено задание 5. В программе создан объект Series, содержащий строковые значения, представляющие названия животных. С помощью свойства dtype был выведен исходный тип данных элементов серии (object). Затем методом astype('category') серия была преобразована в категориальный тип данных (category). После преобразования тип данных был повторно выведен на экран.

5. Определение типа данных Series

Создайте Series из списка `['cat', 'dog', 'rabbit', 'parrot', 'fish']`. Определите тип данных с помощью `.dtype`, затем преобразуйте его в `category` с помощью `.astype()`.

[11]:

```
s = pd.Series(['cat', 'dog', 'rabbit', 'parrot', 'fish'])
print(s.dtype)

s_cat = s.astype('category')
print(s_cat.dtype)
```

object
category

Рисунок 5. Выполнение задания 5

Было выполнено задание 6. В программе создан объект Series, содержащий числовые значения и пропущенные данные (NaN). С помощью метода `isnull()` выполнена проверка элементов на наличие пропусков. Метод `any()` определил, что в серии присутствуют пропущенные значения, и вывел результат `True`. Затем с помощью фильтрации были найдены и выведены индексы элементов, содержащих NaN.

6. Проверка пропущенных значений

Создайте Series с данными `[1.2, np.nan, 3.4, np.nan, 5.6, 6.8]`. Напишите код, который проверяет, есть ли в Series пропущенные значения (NaN), и выведите индексы таких элементов.

[12]:

```
s = pd.Series([1.2, np.nan, 3.4, np.nan, 5.6, 6.8])
print(s.isnull().any())
print(s[s.isnull()].index)
```

True
Index([1, 3], dtype='int64')

Рисунок 6. Выполнение задания 6

Было выполнено задание 7. В программе для объекта Series, содержащего пропущенные значения (NaN), был вычислен средний арифметический показатель всех имеющихся числовых элементов с помощью метода `mean()`.

Затем методом `fillna()` пропущенные значения были заменены найденным средним значением. Полученная серия без пропусков была выведена на экран.

7. Заполнение пропущенных значений

Используйте Series из предыдущего задания и замените все NaN на среднее значение всех непустых элементов. Выведите результат.

[13]:

```
s_filled = s.fillna(s.mean())  
print(s_filled)
```

```
0    1.20  
1    4.25  
2    3.40  
3    4.25  
4    5.60  
5    6.80  
dtype: float64
```

Рисунок 7. Выполнение задания 7

Было выполнено задание 8. В программе созданы две серии Series с частично совпадающими индексами. При выполнении операции сложения `s1 + s2` значения объединяются по индексам. Для совпадающих индексов (b, c, d) производится сложение соответствующих элементов, а для несовпадающих индексов (a, e) формируются пропущенные значения (NaN). Затем методом `fillna(0)` все пропущенные значения заменяются на ноль. Полученная серия выводится на экран.

8. Арифметические операции с Series

Создайте два Series:

- `s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])`
- `s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])`

Выполните сложение `s1 + s2`. Объясните, почему в результате появляются `NaN`, и замените их на `0`.

[14]:

```
s1 = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])
s2 = pd.Series([5, 15, 25, 35], index=['b', 'c', 'd', 'e'])

s_sum = s1 + s2
print(s_sum)

s_filled = s_sum.fillna(0)
print(s_filled)
```

```
a      NaN
b     25.0
c     45.0
d     65.0
e      NaN
dtype: float64
a      0.0
b     25.0
c     45.0
d     65.0
e      0.0
dtype: float64
```

Рисунок 8. Выполнение задания 8

Было выполнено задание 9. В программе создан объект Series, содержащий числовые значения. Затем к каждому элементу серии применена функция `np.sqrt()` с помощью метода `apply()`. В результате для каждого числа вычисляется его квадратный корень, и полученная серия с преобразованными значениями выводится на экран.

9. Применение функции к Series

Создайте Series из чисел `[2, 4, 6, 8, 10]`. Напишите код, который применяет к каждому элементу функцию вычисления квадратного корня с помощью `.apply(np.sqrt)`.

[15]:

```
s = pd.Series([2, 4, 6, 8, 10])
print(s.apply(np.sqrt))
```

```
0    1.414214
1    2.000000
2    2.449490
3    2.828427
4    3.162278
dtype: float64
```

Рисунок 9. Выполнение задания 9

Было выполнено задание 10. В программе создан объект Series, содержащий 20 случайных целых чисел в диапазоне от 50 до 150. Затем были вычислены основные статистические характеристики серии: сумма всех элементов (sum), среднее значение (mean), минимальное (min) и максимальное (max) значения, а также стандартное отклонение (std).

10. Основные статистические методы

Создайте Series из 20 случайных чисел от 50 до 150 (используйте `np.random.randint`). Найдите сумму, среднее, минимальное и максимальное значение. Выведите также стандартное отклонение.

[16]:

```
s = pd.Series(np.random.randint(50, 151, size=20))

print(s.sum())
print(s.mean())
print(s.min())
print(s.max())
print(s.std())
```

```
2280
114.0
56
150
29.790496541859223
```

Рисунок 10. Выполнение задания 10

Было выполнено задание 11. В программе создан временной индекс с помощью `pd.date_range`, который формирует последовательность из 10 дат, начиная с 1 марта 2024 года с шагом в один день. Затем создан объект `Series`, содержащий случайные числовые значения, где в качестве индекса используются даты. После этого выполнена фильтрация данных по диапазону дат с использованием метода `loc`, в результате чего были выведены значения за период с 5 по 8 марта 2024 года включительно.

11. Работа с временными рядами

Создайте `Series`, где индексами будут даты с 1 по 10 марта 2024 года (`pd.date_range(start='2024-03-01', periods=10, freq='D')`), а значениями — случайные числа от 10 до 100. Выберите данные за 5–8 марта.

[17]:

```
dates = pd.date_range(start='2024-03-01', periods=10, freq='D')
s = pd.Series(np.random.randint(10, 101, size=10), index=dates)

print(s.loc['2024-03-05':'2024-03-08'])
```

```
2024-03-05    41
2024-03-06    89
2024-03-07    67
2024-03-08    49
Freq: D, dtype: int32
```

Рисунок 11. Выполнение задания 11

Было выполнено задание 12. В программе создан объект `Series` с повторяющимися индексами. С помощью свойства `is_unique` проверяется уникальность индексов, в результате чего получен логический результат `False`, так как индексы повторяются. Далее выполнена группировка данных по индексам с помощью метода `groupby(level=0)` и вычисление суммы значений для каждого уникального индекса. В результате повторяющиеся элементы были агрегированы, и получена новая серия с суммированными значениями по каждому индексу.

12. Проверка уникальности индексов

Создайте Series с индексами ['A', 'B', 'A', 'C', 'D', 'B'] и значениями [10, 20, 30, 40, 50, 60]. Проверьте, являются ли индексы уникальными. Если нет, сгруппируйте повторяющиеся индексы и сложите их значения.

[18]:

```
s = pd.Series([10, 20, 30, 40, 50, 60], index=['A', 'B', 'A', 'C', 'D', 'B'])

print(s.index.is_unique)

s_grouped = s.groupby(level=0).sum()
print(s_grouped)
```

```
False
A      40
B      80
C      40
D      50
dtype: int64
```

Рисунок 12. Выполнение задания 12

Было выполнено задание 13. В программе создан объект Series, индексами которого являются строки, представляющие даты. Затем индекс серии был преобразован в тип datetime с помощью функции `pd.to_datetime()`. После преобразования индекс стал временным типом данных, что позволяет выполнять операции с датами (например, сортировку, фильтрацию по времени и анализ временных рядов). В конце на экран выведен обновлённый индекс серии.

13. Преобразование строковых дат в DatetimeIndex

Создайте Series, где индексами будут строки ['2024-03-10', '2024-03-11', '2024-03-12'], а значениями — [100, 200, 300]. Преобразуйте индексы в DatetimeIndex и выведите тип данных индекса.

[19]:

```
s = pd.Series([100, 200, 300], index=['2024-03-10', '2024-03-11', '2024-03-12'])
s.index = pd.to_datetime(s.index)

print(s.index)
```

```
DatetimeIndex(['2024-03-10', '2024-03-11', '2024-03-12'], dtype='datetime64[ns]', freq=None)
```

Рисунок 13. Выполнение задания 13

Было выполнено задание 14. В программе сначала создаётся CSV-файл `data.csv`, содержащий данные о дате и цене. Затем файл считывается с помощью функции `pd.read_csv()`, при этом столбец «Дата» используется в качестве индекса (`index_col='Дата'`). После загрузки данных из `DataFrame` выбирается столбец «Цена», который преобразуется в объект `Series`. В результате на экран выводится серия значений цен, индексированных по датам.

14. Чтение данных из CSV-файла

Создайте CSV-файл `data.csv` со следующими данными:

```
Дата, Цена
2024-03-01, 100
2024-03-02, 110
2024-03-03, 105
2024-03-04, 120
2024-03-05, 115
```

Прочитайте файл и создайте `Series`, используя «Дата» в качестве индекса.

[20]:

```
with open('data.csv', 'w', encoding='utf-8') as f:
    f.write("Дата,Цена\n2024-03-01,100\n2024-03-02,110\n2024-03-03,105\n2024-03-04,120\n2024-03-05,115\n")

df = pd.read_csv('data.csv', index_col='Дата')
s = df['Цена']
print(s)
```

```
Дата
2024-03-01    100
2024-03-02    110
2024-03-03    105
2024-03-04    120
2024-03-05    115
Name: Цена, dtype: int64
```

Рисунок 14. Выполнение задания 14

Было выполнено задание 15. В программе создаётся временной ряд с помощью `pd.date_range`, формирующий последовательность из 30 дат, начиная с 1 марта 2024 года. Затем создаётся объект `Series` со случайными значениями в диапазоне от 50 до 150, где индексом являются даты. Далее с использованием библиотеки `matplotlib.pyplot` строится линейный график значений серии. На

графике задаются заголовок, подписи осей и включается сетка для удобства визуального анализа данных. В результате отображается график изменения значений во времени.

15. Построение графика на основе Series

Создайте Series, где индексами будут даты с 1 по 30 марта 2024 года, а значениями – случайные числа от 50 до 150. Постройте график значений с помощью `matplotlib`. Добавьте заголовок, подписи осей и сетку.

```
import matplotlib.pyplot as plt

dates = pd.date_range(start='2024-03-01', periods=30, freq='D')
s = pd.Series(np.random.randint(50, 151, size=30), index=dates)

plt.figure(figsize=(10, 5))
plt.plot(s)
plt.title("График значений Series")
plt.xlabel("Дата")
plt.ylabel("Значение")
plt.grid()
plt.show()
```



Рисунок 15. Выполнение задания 15

Было выполнено индивидуальное задание по варианту 12. В программе создаётся CSV-файл с данными о спросе на товар по датам. Затем данные считываются в DataFrame с использованием столбца «Дата» в качестве индекса, после чего индекс преобразуется в тип `datetime` для работы с временными рядами. Из столбца «Спрос» формируется объект Series. Далее вычисляется скользящее среднее с окном 3 с помощью метода `rolling(window=3).mean()`, что позволяет сгладить колебания данных. После этого строится график, на котором

отображаются исходные значения спроса и сглаженный ряд. График дополняется легендой, подписями осей, заголовком и сеткой.

12. Индивидуальное задание

Создайте CSV-файл `demand.csv` со следующими данными:

```
Дата, Спрос
2024-09-01, 120
2024-09-02, 135
2024-09-03, 150
2024-09-04, 170
2024-09-05, 160
```

Прочитайте файл, установите `DatetimeIndex`, вычислите скользящее среднее (`rolling(3).mean()`) и постройте два графика: фактического спроса и его сглаженного значения.

```
with open('demand.csv', 'w', encoding='utf-8') as f:
    f.write("Дата,Спрос\n2024-09-01,120\n2024-09-02,135\n2024-09-03,150\n2024-09-04,170\n2024-09-05,160")

df = pd.read_csv('demand.csv', index_col='Дата')
df.index = pd.to_datetime(df.index)
s = df['Спрос']

s_ma = s.rolling(window=3).mean()

plt.figure(figsize=(10, 5))
plt.plot(s, label="Фактический спрос", marker="o")
plt.plot(s_ma, label="Скользящее среднее (3)", linestyle="--", color="red")
plt.title("Динамика спроса на товар")
plt.xlabel("Дата")
plt.ylabel("Спрос")
plt.legend()
plt.grid()
plt.show()
```

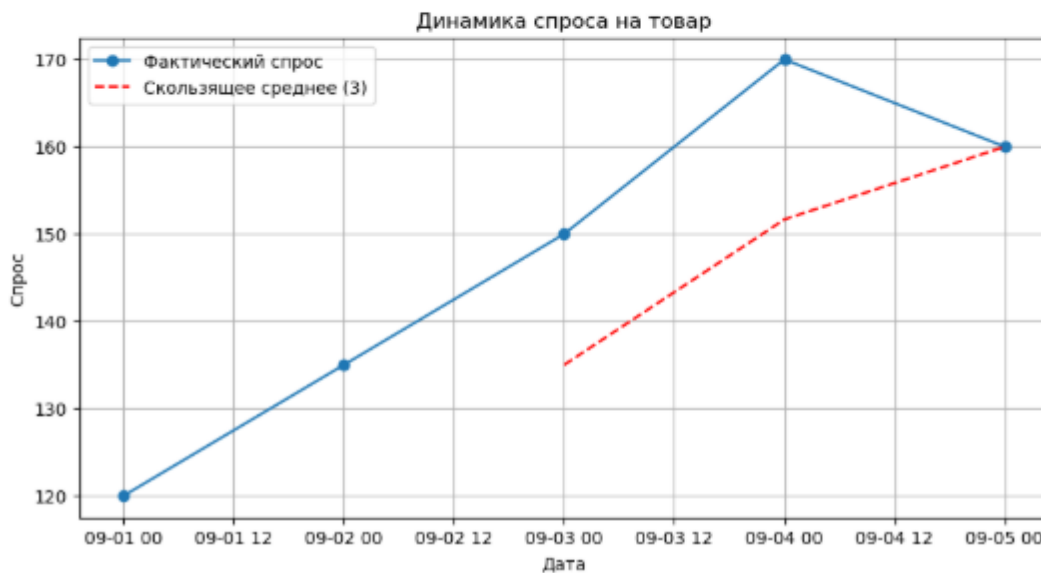


Рисунок 16. Выполнение индивидуального задания по варианту 12

Ответы на контрольные вопросы

1. Что такое `pandas.Series` и чем она отличается от списка в Python?

`pandas.Series` - это одномерный массив с индексами из библиотеки `pandas`. От списка Python он отличается тем, что имеет индексы, поддерживает векторные операции, умеет работать с NaN и содержит методы для анализа данных

2. Какие типы данных можно использовать для создания Series?

Для создания `pandas.Series` можно использовать списки Python, массивы NumPy, словари, скалярные значения и другие объекты Series.

3. Как задать индексы при создании Series?

Индексы в Series задаются через параметр `index` при её создании. В этот параметр передаётся список меток, которые будут соответствовать каждому элементу данных.

4. Каким образом можно обратиться к элементу Series по его индексу?

Обратиться к элементу Series по индексу можно через квадратные скобки, указав имя индекса. Например, `s['a']` вернёт значение, которое соответствует индексу 'a'. Также можно использовать `.loc[]` для явного обращения по метке индекса, например `s.loc['a']`.

5. В чём разница между `.iloc[]` и `.loc[]` при индексации Series?

`.loc[]` используется для доступа к элементам по именам индексов (меткам), например `s.loc['a']`. `.iloc[]` используется для доступа по числовой позиции элемента, то есть по его порядковому номеру, например `s.iloc[0]`.

6. Как использовать логическую индексацию в Series?

Логическая индексация в Series используется через условие внутри квадратных скобок. Например, `s[s > 10]` возвращает только те элементы, которые больше 10. Условие создаёт булеву маску, и `pandas` отбирает элементы, где значение True.

7. Какие методы можно использовать для просмотра первых и последних элементов Series?

Для просмотра первых элементов используется метод `head()`, а для последних — метод `tail()`. Например, `s.head()` покажет первые 5 элементов, а

`s.tail()` — последние 5. Можно также указать количество элементов, например `s.head(3)` или `s.tail(2)`.

8. Как проверить тип данных элементов Series?

Тип данных элементов Series проверяется через свойство `dtype`.

9. Каким способом можно изменить тип данных Series?

Тип данных Series изменяется с помощью метода `astype()`, который возвращает новую серию с указанным типом данных.

10. Как проверить наличие пропущенных значений в Series?

Пропущенные значения в Series проверяются с помощью методов `isnull()` или `isna()`. Они возвращают булеву серию, где `True` означает наличие пропуска (NaN), а `False` — обычное значение.

11. Какие методы используются для заполнения пропущенных значений в Series?

Для заполнения пропущенных значений в Series используется метод `fillna()`.

12. Чем отличается метод `.fillna()` от `.dropna()`?

`fillna()` заменяет пропущенные значения (NaN) на заданное значение, например число, среднее или другое вычисленное значение. `dropna()` удаляет элементы с пропущенными значениями из Series, то есть полностью исключает их из данных

13. Какие математические операции можно выполнять с Series?

С Series можно выполнять обычные математические операции: сложение, вычитание, умножение и деление. Также поддерживаются операции сравнения (`>`, `<`, `==` и т.д.) и поэлементные вычисления с числами или другими Series. Все операции выполняются поэлементно и автоматически выравниваются по индексам.

14. В чём преимущество векторизованных операций по сравнению с циклами Python?

Векторизированные операции выполняются сразу над всеми элементами массива без использования явных циклов Python. Их преимущество в том, что они работают значительно быстрее, так как реализованы на уровне оптимизированных библиотек (NumPy), а не интерпретатора Python. Также код становится более коротким и читаемым.

15. Как применить пользовательскую функцию к каждому элементу Series?

Пользовательскую функцию к каждому элементу Series применяют с помощью метода `apply()`

16. Какие агрегирующие функции доступны в Series?

В Series доступны основные агрегирующие функции: `sum()` для суммы, `mean()` для среднего значения, `min()` и `max()` для нахождения минимума и максимума, а также `std()` для стандартного отклонения.

17. Как узнать минимальное, максимальное, среднее и стандартное отклонение Series?

Минимальное значение находится методом `min()`, максимальное `max()`, среднее `mean()`, стандартное отклонение `std()`.

18. Как сортировать Series по значениям и по индексам?

Series можно сортировать двумя способами. По значениям используется метод `sort_values()`, например `s.sort_values()`. По индексам - метод `sort_index()`, например `s.sort_index()`. В обоих случаях можно указать порядок сортировки через параметр `ascending`.

19. Как проверить, являются ли индексы Series уникальными?

Уникальность индексов в Series проверяется через свойство `index.is_unique`. Оно возвращает `True`, если все индексы уникальны, и `False`, если есть повторы.

20. Как сбросить индексы Series и сделать их числовыми?

Сбросить индексы можно методом `reset_index()`. Например, `s.reset_index(drop=True)` заменяет текущие индексы на числовые 0, 1, 2 и т.д. Параметр `drop=True` означает, что старые индексы не сохраняются в виде отдельного столбца.

21. Как можно задать новый индекс в Series?

Новый индекс в Series можно задать напрямую, присвоив список значений свойству `index`, например `s.index = ['a', 'b', 'c']`. Также индекс можно задать при создании Series через параметр `index`.

22. Как работать с временными рядами в Series?

С временными рядами в Series работают через индекс типа `datetime`, который позволяет воспринимать значения как даты и время. Сначала строки с датами преобразуют в формат времени с помощью `pd.to_datetime()`, после чего этот индекс можно использовать для анализа данных по времени.

23. Как преобразовать строковые даты в формат DatetimeIndex?

Строковые даты преобразуются в формат `DatetimeIndex` с помощью функции `pd.to_datetime()`. После преобразования они начинают восприниматься как временные значения, что позволяет использовать их для анализа, фильтрации и работы с временными рядами.

24. Каким образом можно выбрать данные за определённый временной диапазон?

Данные за определённый временной диапазон в Series выбираются с помощью среза по датам, если индекс имеет тип `DatetimeIndex`. При этом указываются начальная и конечная даты, и pandas возвращает все значения, попадающие в этот промежуток времени

25. Как загрузить данные из CSV-файла в Series?

Данные из CSV-файла загружаются с помощью функции `pd.read_csv()`.

26. Как установить один из столбцов CSV-файла в качестве индекса Series?

Один из столбцов CSV-файла устанавливается в качестве индекса с помощью параметра `index_col` в функции `pd.read_csv()`. В этом случае указанный столбец становится индексом, а данные из других столбцов могут использоваться как `Series`

27. Для чего используется метод `.rolling().mean()` в `Series`?

Метод `.rolling().mean()` используется для вычисления скользящего среднего в `Series`. Он берёт заданное окно значений и вычисляет среднее внутри этого окна, постепенно «скользя» по данным. Это применяется для сглаживания временных рядов и уменьшения случайных колебаний.

28. Как работает метод `.pct_change()`? Какие задачи он решает?

Метод `.pct_change()` вычисляет процентное изменение между текущим и предыдущим элементом в `Series`. Он показывает, на сколько процентов изменилось значение относительно предыдущего. Этот метод используется для анализа динамики данных, например роста или падения показателей во времени.

29. В каких ситуациях полезно использовать `.rolling()` и `.pct_change()`?

Метод `.rolling()` полезен при анализе временных рядов, когда нужно сгладить данные и увидеть общий тренд без случайных колебаний. Метод `.pct_change()` используется для оценки динамики изменений, например роста или падения значений между соседними периодами.

30. Почему `NaN` могут появляться в `Series`, и как с ними работать?

`NaN` могут появляться в `Series` по нескольким причинам: из-за отсутствующих данных в исходном наборе, при несовпадении индексов в операциях между `Series`, при делении на ноль или при использовании функций, которые не могут вычислить значение (например, скользящие окна в начале ряда).

Работать с `NaN` можно разными способами: их можно удалить с помощью `dropna()`, заменить значениями с помощью `fillna()`, либо восстановить с помощью интерполяции через `interpolate()`.

Вывод: В ходе лабораторной работы были изучены основные возможности `pandas.Series`, включая создание, индексацию, фильтрацию, обработку пропусков, выполнение математических и статистических операций, работу с временными рядами и CSV-файлами. Получены базовые навыки анализа и преобразования данных в `pandas`.